



Systèmes Temps Réels



Classe : 3 LCE SE
Code UE: UEF540

Enseignant : Adel KACEM

E-mail : adel.kacem@gmail.com

1

Systèmes Temps Réels

Objectifs

- Initier l'étudiant à la notion de systèmes temps réel,
- Apprendre les spécificités et les contraintes de ce type de systèmes,
- Inculquer l'architecture et le fonctionnement d'un noyau temps réel (Tâche, interruption, etc.),
- Découvrir les outils de synchronisation et de communication entre tâches (Mutex, sémaphore, inversion de priorité, etc),
- Comprendre la problématique du multitâche et de connaître la notion d'ordonnancement des tâches dans contexte temps réel.

2

Systèmes Temps Réels

Plan

- I. Exécutifs Temps Réels.
 1. Introduction
 2. Définitions
 3. Services RTOS
 4. Quelques standards de RTOS
- II. Partage de ressources.
 1. Introduction
 2. Les tâches dépendantes
 3. Mutex et Sémaphore
 4. Gestion de la mémoire et du temps
 5. Les interruptions

3

Systèmes Temps Réels

I. Exécutifs Temps Réels.

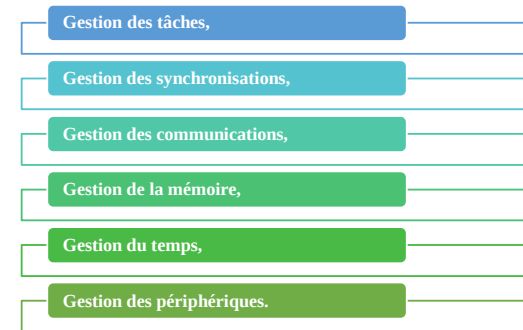
4

1.Introduction: Pourquoi un RTOS?

- Dans un contexte informatique, un système temps réel est représenté par une association **logiciel** et **matériel**.
- Le rôle de la partie logiciel :
 - ✓ La gestion adéquate des ressources matérielles dont l'objectif est de remplir certaines tâches dans des limites temporelles bien définie.
 - ✓ Le RTOS offre des services au(x) logiciel(s) d'application
- Ces services sont basés sur les ressources disponibles au niveau du matériel.

5

1.Introduction: Pourquoi un RTOS?



6

1.Introduction: Généralités

- Un Système d'exploitation temps réel est un OS avec des caractéristiques permettant de supporter le RT.
- Le RTOS est un programme dont le rôle est:
 - ✓ Ordonnancer l'exécution des tâches,
 - ✓ Gérer les ressources du système,
 - ✓ Fournir les éléments de base pour développer les applications.
- Le RTOS est organisé autour du noyau.
- Le noyau fournit les algorithmes permettant de faire l'ordonnancement et la gestion des ressources et des services tels que la gestion des systèmes de fichier, l'accès réseau, et tout autre service requis par l'application).

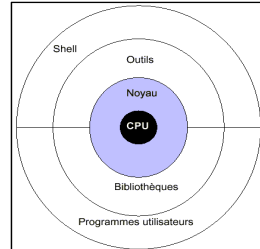
1.Introduction: Généralités

- Pour respecter les contraintes temporelles, il est nécessaire que:
 - ✓ les différents services et algorithmes utilisés s'exécutent en temps borné:¹
 - ⇒ le RTOS doit être déterministe et préemptif pour donner la main à la tâche de plus forte priorité prête.
 - ⇒ Caractéristique importante d'un RTOS = temps de réponse prévisible à un stimulus externe.
 - ✓ Dans le cas où un périphérique génère une interruption, le RTOS doit répondre et démarrer le service à l'intérieur d'une période de temps connue.
 - ✓ Un OS est un système temps réel lorsque le temps de réponse à une interruption (le changement de contexte) est garanti à l'intérieur d'une période de 10 µs.

¹ <https://moodle.insa-toulouse.fr/>

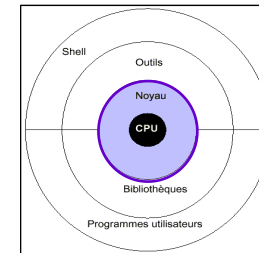
2.Définitions: Structure d'un RTOS

- Un ROTS est organisé autour d'un noyau qui fournit les algorithmes minimum pour l'ordonnancement et la gestion des ressources de services (gestion systèmes de fichier, accès réseau, ...).



2.Définitions: Structure d'un RTOS

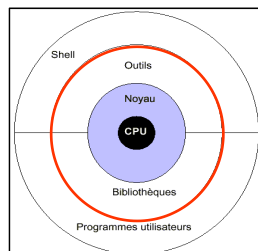
Le noyau gère:



- les périphériques
- L'exécution des programmes (processus)
- la mémoire attribuée à chaque processus
- l'ordonnancement des processus
- la synchronisation et communication entre processus
- la gestion des protocoles réseau
- logiciel compact, de petite taille (moins de 10 Ko)

2.Définitions: Structure d'un RTOS

L'exécutif construit sur le noyau et contient des fonctionnalités comme:



La gestion des entrées/sorties

Le mécanismes de coopération évolués (construits à partir de ceux du noyau)

La gestion d'un système de fichiers

La facilités (Exemple: connexion distante).

2.Définitions: rôle d'un exécutif TR

- Un exécutif temps réel peut être:²
 - ❖ Event-driven systems switch between tasks based on their priorities.
 - ❖ Time sharing : systems switch the task based on clock interrupts.

² [wikipedia.org](https://fr.wikipedia.org/wiki/Ex%C3%A9cutif_temps_r%C3%A9el)

2.Définitions: rôle d'un exécutif TR

- Les appels à l'exécutif sont la conséquence :³
 - ❖ **des occurrences d'événements issus du procédé:**
l'appel à l'exécutif est effectué par la procédure de réception et de traitement de l'interruption matérielle associée à ces événements.
 - ❖ **du temps**
l'appel à l'exécutif est provoqué par l'interruption régulièrement engendrée par une horloge temps réel équipant le calculateur.
 - ❖ **des tâches elles-mêmes**
c'est le cas lorsqu'une tâche requiert des services offerts par l'exécutif.

³techniques-ingenieur.fr

2.Définitions: rôle d'un exécutif TR

- L'exécutif TR, offre des services accessibles par l'utilisateur dans ses tâches, de différentes natures gestion des tâches :
 - ❖ Activation, suspension, reprise, terminaison forcée ...,
 - ❖ Gestion des événements matériels (interruptions),
 - ❖ Synchronisation : émission/réception par les tâches de signaux « internes ».
- L'exécutif offre des services de génération et d'attente de ces « événements logiciels ».

2.Définitions: rôle d'un exécutif TR

- Un exécutif temps réel permet d'ordonnancer les exécutions des tâches et de protéger l'accès aux ressources partagées. Il joue un rôle centralisateur :³
 - ❖ interface qui aiguille les événements reçus du procédé vers les tâches qui les attendent,
 - ❖ déclenche le réveil des tâches en attente d'un délai ou d'une heure de démarrage,
 - ❖ reçoit et retransmet des signaux de synchronisation ou des données entre des tâches asynchrones.

³techniques-ingenieur.fr

2.Définitions: rôle d'un exécutif TR

- Pour illustrer la différence entre un RTOS et un système d'exploitation standard, soit l'exemple suivant:
- Vous êtes dans un jeu vidéo dans lequel chacune de vos actions déclenche l'exécution d'un programme.
 - ❖ Si le jeu est géré par un RTOS, vos actions vont être exécutées avec un léger retard, connu et borné.
 - ❖ Si le jeu est géré par un OS standard, ce retard n'est pas fiable, les effets des actions risquent d'être désynchronisés.
- Pour garantir le temps de réponse, les programmes TR et leurs OS doivent gérer en priorité l'actualisation des échéances des actions. Cela va se traduire par exemple par une qualité d'image moindre éventuellement.

3.Services RTOS: Gestion des tâches

Types d'ordonnanceurs:

- Ordonnanceurs non préemptifs (séquenceurs)
 - ❖ Jouent une séquence d'exécution générée statiquement (non préemptif)
 - ❖ Noyaux propriétaires essentiellement (non commerciaux), hormis OsekTime (automobile) ⁵
- Ordonnanceurs préemptifs
 - ❖ Ordonnanceurs à priorités
 - Priorité par tâche, allocation du processeur à la tâche la plus prioritaire, préemptif
 - Quasi-totalité des exécutifs temps-réel
 - ❖ De type temps-partagé: Allocation du processeur par tranche de temps (préemptif)

⁵ <http://www.mnis.fr/>

3.Services RTOS: Gestion des tâches

- Exigence du respect des contraintes pas systématique pour toutes les tâches.
- Les tâches critiques doivent impérativement être exécutées dans des fenêtres temporelles conditionnées par les dynamiques du procédé.
 - ❖ le non-respect des contraintes des tâches critiques peut compromettre la stabilité du procédé, sa sécurité ou celle de son environnement.
 - ❖ celui des tâches non critiques peut altérer certaines prestations du procédé ou en dégrader les performances

3.Services RTOS: Gestion des tâches

- Les exécutions des tâches doivent respecter des contraintes temporelles dictées par les dynamiques du procédé qu'elles pilotent ou contrôlent.
- Ordonnancer des tâches consiste à:
 1. Déterminer la politique d'allocation des tâches sur les supports d'exécution (processeurs, contrôleurs) pour garantir ces contraintes temporelles.
 2. Puis, implémenter cette politique : déterminer le mécanisme à intégrer dans l'exécutif qui lui permettra de décider de l'ordre des tâches lors de l'évolution de l'application.

3.Services RTOS: Gestion des tâches

- L'ordonnancement est une politique valide doit garantir ces contraintes pour toutes les combinaisons possibles d'événements.
- On peut tenter de garantir ce respect des contraintes de deux façons différentes:
 1. Déterminer hors-ligne une séquence spécifique des exécutions des tâches qui soit valide pour l'application concernée. Ceci peut exiger l'énumération de toutes les combinaisons possibles d'événements et la recherche associée d'un ordre d'exécution des tâches.
 2. Elaborer des algorithmes d'ordonnancement dont on puisse prouver qu'ils sont optimaux pour certains modèles d'applications et de contraintes, puis faire rejoindre l'application traitée à l'un de ces modèles.

3.Services RTOS: Gestion des tâches

Deux grandes classes d'ordonnancement:

- **Ordonnancement « statique »**
 - Effectue le calcul de la séquence d'allocation des processeurs à partir de la connaissance complète de toutes les caractéristiques des tâches, avant le lancement de l'application.
 - Hypothèse réaliste pour de nombreuses applications :
 - Nombre « raisonnable » de tâches aux échéances différentes (moins de 100),
 - Taux d'occupation des processeurs élevé,
 - Aucun dépassement d'échéance autorisé.

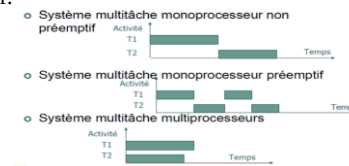
3.Services RTOS: Gestion des tâches

Deux grandes classes d'ordonnancement:

- **Ordonnancement « dynamique »**
 - Alloue les processeurs à partir de la seule connaissance des caractéristiques des tâches en cours d'exécution et de celles en demande d'exécution au moment du choix.
 - Fixe dynamiquement son choix en ignorant l'existence des tâches futures.
 - La séquence construite varie dans le temps en fonction des occurrences d'événements, donc des réveils et arrêts des tâches de l'application.

3.Services RTOS: Gestion des tâches

- Niveaux de parallélisme:⁴
 - ✓ Parallélisme réel : Systèmes multi-processeurs uniquement
 - ✓ Pseudo-parallélisme : le processeur exécute successivement plusieurs traitements.
- Les traitements sont exécutés en séquence : ordonnancement non préemptif,
- Un traitement peut être interrompu par un autre traitement : ordonnancement préemptif.



⁴ <https://moodle.insa-toulouse.fr/>

3.Services RTOS: relations inter-tâches

- Relations inter-tâches de deux types:
 - ✓ Précédence (synchronisation par événement)
 - ❖ Notion d'ordre entre les traitements.
 - ✓ Partage de variables/ressources
 - ❖ Échanges d'informations ou de résultats.
- Primitives de synchronisation/communication offertes par les exécutifs temps-réel.

3.Services RTOS: relations inter-tâches

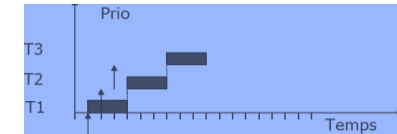
- Un **sémaphore** est une variable partagée par différents « acteurs », qui garantit que ceux-ci ne peuvent y accéder que de façon séquentielle à travers des opérations atomiques, et constitue la méthode utilisée couramment pour restreindre l'accès à des ressources partagées.²
- Exemple de synchronisation par sémaphores:
- Soit T1, T2 et T3 trois tâches tel que T1 arrive en 1, de T2 en 2, T3 en 3 avec des priorités : $prio(T1) < prio(T2) < prio(T3)$; et les durées des A_i de 3;⁶

² [wikipedia.org](https://fr.wikipedia.org/wiki/S%C3%A9maphore)

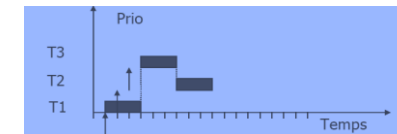
⁶ <https://sites.uclouvain.be/>

3.Services RTOS: relations inter-tâches

- Cas d'une file gérée en FIFO



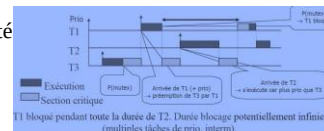
- Cas d'une file gérée par priorité croissante



3.Services RTOS: relations inter-tâches

Partage de ressources

- Implantation d'une section critique,
- Utilisation de sémaphores (attente passive),
- Principe : Request(v) $\rightarrow$ P(s_v); Release(v) $\rightarrow$ V(s_v);
- Sémaphores binaires,
- Sémaphores à compteurs,
- Sémaphores d'exclusion mutuelle (mutex) : sémaphores avec en plus les propriétés
- Problème d'inversion de priorité



<https://moodle.insa-toulouse.fr/>

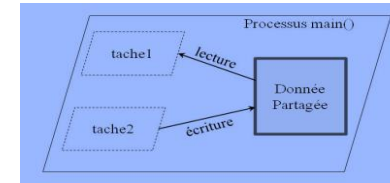
4. Quelques standards de RTOS

- POSIX** ("Portable Operating Systems Interface") is a standard for the function calls (API) of UNIX-like general purpose operating systems.
- Some specifications on real-time primitives too. Its definition of real time is quite loose: ability of the OS to provide a required level of service in a bounded response time.
- Many of the real-time POSIX extensions have already been implemented in RTLinux.

Systèmes Temps Réels

II. Partage de ressources.

1.Introduction:



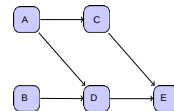
33

34

2.Les tâches dépendantes

2.1 Contraintes de précédence

- Problématique : Une tâche j doit s'exécuter après i
- Les contraintes de précédence sur l'ordre d'exécution des tâches les unes par rapport aux autres sont généralement décrites par un graphe orienté :
- $i < j$ indique que la tâche i est un prédécesseur de j
- $i \rightarrow j$ indique que la tâche i est un prédécesseur immédiat de j



35

2.Les tâches dépendantes

2.1 Contraintes de précédence

- Ici, seulement précédence simple : Les tâches dépendantes ont la même période.
- Le principe de l'établissement de l'ordonnancement :
 - ❖ Transformer l'ensemble des tâches dépendantes en un ensemble de tâches indépendantes que l'on ordonnancera par un algorithme classique.
 - ❖ Par des modifications des paramètres des tâches : si $j \rightarrow i$ alors la règle de transformation doit respecter :
 - ✓ $r_j \leq r_i$
 - ✓ $P_i < P_j$

36

2.Les tâches dépendantes

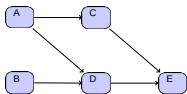
2.2 Contraintes de précédence avec RM (Rate Monotonic)

- Contraintes de précédence et Rate Monotonic : la transformation s'opère hors ligne sur la date de réveil et sur les délais critiques
 - $r_i^* = \max\{r_i, r_j^*\}$ pour tous les j tels que $j \rightarrow i$
 - si $i \rightarrow j$ alors $P_i > P_j$ dans le respect de la règle d'affectation des priorités par Rate Monotonic.

37

2.Les tâches dépendantes

2.2 Contraintes de précédence avec RM (Rate Monotonic)

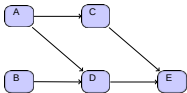


Tâche	Capacité	Arrivée	r_i^*	P_i
A	1	0		
B	2	5		
C	2	0		
D	1	0		
E	3	0		

38

2.Les tâches dépendantes

2.2 Contraintes de précédence avec RM (Rate Monotonic)

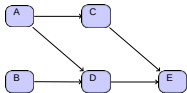


Tâche	Capacité	Arrivée	r_i^*	P_i
A	1	0	0	
B	2	5	5	
C	2	0	0	
D	1	0	5	
E	3	0	5	

39

2.Les tâches dépendantes

2.2 Contraintes de précédence avec RM (Rate Monotonic)



Tâche	Capacité	Arrivée	r_i^*	P_i
A	1	0	0	1
B	2	5	5	1
C	2	0	0	2
D	1	0	5	2
E	3	0	5	3

40

2.Les tâches dépendantes

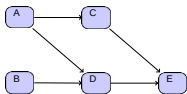
2.3 Contraintes de précédence avec DM (Deadline Monotonic)

- Contraintes de précédence et Deadline Monotonic : la transformation s'opère hors ligne sur la date de réveil et sur les délais critiques.
 - ❖ $r_i^* = \max\{r_i, r_j^*\}$ pour tous les j tels que $j \rightarrow i$
 - ❖ $D_i^* = \max\{D_i, D_j^*\}$ pour tous les j tels que $j \rightarrow i$ (ne correspond pas aux délais critiques réels)
- Si $i \rightarrow j$ alors $P_i > P_j$ dans le respect de la règle d'affectation des priorités par DM.

41

2.Les tâches dépendantes

2.3 Contraintes de précédence avec DM (Deadline Monotonic)

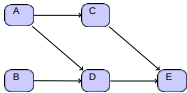


Tâche	Capacité	Arrivée	Délai	r_i^*	D_i^*	P_i
A	1	0	5	0		
B	2	5	7	5		
C	2	0	4	0		
D	1	0	5	5		
E	3	0	6	5		

42

2.Les tâches dépendantes

2.3 Contraintes de précédence avec DM (Deadline Monotonic)

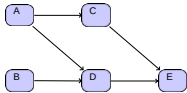


Tâche	Capacité	Arrivée	Délai	r_i^*	D_i^*	P_i
A	1	0	5	0	5	
B	2	5	7	5	7	
C	2	0	4	0	5	
D	1	0	5	5	7	
E	3	0	6	5	7	

43

2.Les tâches dépendantes

2.3 Contraintes de précédence avec DM (Deadline Monotonic)



Tâche	Capacité	Arrivée	Délai	r_i^*	D_i^*	P_i
A	1	0	5	0	5	1
B	2	5	7	5	7	3
C	2	0	4	0	5	2
D	1	0	5	5	7	4
E	3	0	6	5	7	5

44

2.Les tâches dépendantes

2.4 Contraintes de précedence avec EDF (Earliest Deadline First)

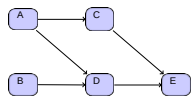
- Modification des échéances de façon à ce qu'une tâche ait toujours un « d_i » inférieur à celui de ses successeurs (algorithme de Chetto et al.)
- Une tâche ne doit être activable que si tous ses prédécesseurs ont terminé leur exécution.
- Modification de la date de réveil et de l'échéance
 - ❖ $r_i^* = \max\{r_i, \max\{r_j^* + C_j\} \text{ pour tous les } j \text{ tels que } j \rightarrow i\}$
 - ❖ $d_i^* = \min\{d_i, \min\{d_j^* - C_j\} \text{ pour tous les } j \text{ tels que } i \rightarrow j\}$
- On itère sur les prédécesseurs et successeurs immédiats.
- On commence les calculs par les tâches qui n'ont pas de prédécesseurs pour le calcul des r et par les tâches qui n'ont pas de successeur pour le calcul des d.

$de^* = \min\{de, \min\{\text{ensemble vide}\}\}$
 $= \min\{de=12\}$
 $= 12$
 $dd^* = \min\{dd, \min\{de^* - Ce\}\}$
 $= \{10, \min\{12-3\}\}$
 $= \{10, 9\} = 9$

$dc^* = \min\{dc, \min\{de^* - Ce\}\}$
 $= \min\{5, 12-3=9\}$
 $= 5$
 $db^* = \min\{db, \min\{dd^* - Cd\}\}$
 $= \min\{7, 10-1\} = 7$
 $da^* = \min\{da, \min\{dc^* - Cc, dd^* - Cd\}\}$
 $= \min\{5, \min\{5-2=3, 10-1=9\}\}$
 $= 3$

2.Les tâches dépendantes

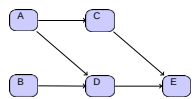
2.4 Contraintes de précedence avec EDF (Earliest Deadline First)



Tâche	Capacité	Arrivée	Délai	r_i^*	d_i^*
A	1	0	5	0	
B	2	5	7	5	
C	2	0	5	1	
D	1	0	10	7	
E	3	0	12	8	

2.Les tâches dépendantes

2.4 Contraintes de précedence avec EDF (Earliest Deadline First)

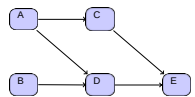


Tâche	Capacité	Arrivée	Délai	r_i^*	d_i^*
A	1	0	5	$rA^*=0$	$de^*=12$
B	2	5	7	$rB^*=5$	$dd^*=10$
C	2	0	5	$rC^*=1$	$dc^*=5$
D	1	0	10	$rD^*=7$	$db^*=7$
E	3	0	12	$rE^*=8$	$da^*=3$

$rd^* = \max\{0, \max\{0+1, 5+2\}\}$
 $= \max\{0, 7\}$
 $= 7$
 $re^* = \max\{re, \max\{rc^* + Cc, rd^* + Cd\}\}$
 $= \max\{0, \max\{1+2=3, 7+1=8\}\}$
 $= \max\{0, 8\}$
 $= 8$

2.Les tâches dépendantes

2.4 Contraintes de précedence avec EDF (Earliest Deadline First)



Tâche	Capacité	Arrivée	Délai	r_i^*	d_i^*
A	1	0	5	0	3
B	2	5	7	5	7
C	2	0	5	1	5
D	1	0	10	7	9
E	3	0	12	8	12

2. Les tâches dépendantes

2.5 Exemple de partage de données

- Le partage de données entre les tâches posent les mêmes problèmes que le partage de données avec les interruptions,
- Prenons deux tâches f1 et f2 :

```
int a = 0;
char t[255];
void f1() {
    t[a] = data1;
    a++;
}
void f2() {
    t[a] = data2;
    a++;
}
```

49

2. Les tâches dépendantes

2.5 Exemple de partage de données

- Prenons le cas où f1 est préemptée par f2 :

```
t[a] = data1; // t[0];

a++;
// a = 2 maintenant
```

```
t[a] = data2;
// t[0] est écrasé!
a++;
// a = 1 maintenant
```

- Au lieu d'écrire les deux données l'une après l'autre, la valeur de data1 est perdue alors que t[1] contiendra une valeur aléatoire.

50

2. Les tâches dépendantes

2.5 Exemple de partage de données

- Cas d'un périphérique réseau avec des registres mappés en mémoire. Le registre 0xABC0 permet de placer la donnée à envoyer.

```
void send(int data) {
    *0xABC = data;
    *0xABC4 = 1;
}
```

51

2. Les tâches dépendantes

2.6 Exemple de ressource partagée

- Étudions le cas de l'exécution simultanée de cette fonction par deux tâches. La première tâche appelle send avec data = 42 :

```
*0xABC0 = 42
```

- La tâche est préemptée. La seconde tâche appelle send avec data = 10 :

```
*0xABC0 = 10
*0xABC4 = 1
```

- 10 est envoyé. La tâche 1 reprend la main :

```
*0xABC4 = 1
```

- 10 (au lieu de 42) est de nouveau envoyé.

52

2. Les tâches dépendantes

2.7 Comment s'en protéger ?

- Ne pas utiliser de variables globales ou de ressources partagées.
- Utiliser des accès atomiques (sans pouvoir être interrompues avant la fin de leur déroulement).
- Placer des accès aux ressources partagées dans des sections critiques.
- Une fonction pouvant être appelée simultanément depuis deux contextes de tâches différentes est dite réentrante.

53

3. Mutex et Sémaphore

3.2 Fonctionnement d'un mutex

- Nécessite une instruction assembleur permettant un accès en lecture et en écriture en une instruction :
- test_and_set affecte le registre d'état en fonction de la valeur du registre et affecte la valeur 1 au registre. On peut développer la fonction lock à partir de là :

```
void lock(mutex_t *m) {
    while (test_and_set(*m))
        schedule();
}
void unlock(mutex_t *m) {
    *m = 0;
    schedule();
}
```

3. Mutex et Sémaphore

3.1 Partage de ressources critiques

- Une ressource critique ne peut :
 - ❖ être utilisée simultanément par plusieurs tâches,
 - ❖ être réquisitionnée par une autre tâche.
 - Notion de section critique :
 - ❖ séquence d'instructions pendant lesquelles on utilise une ressource critique,
 - ❖ sans problème dans le cas d'un ordonnancement non préemptif, mais c'est rarement le cas dans un environnement temps réel.
- ⇒ évaluation du temps de réponse très difficile, sinon impossible.

54

3. Mutex et Sémaphore

3.2 Fonctionnement d'un mutex

- Un peu mieux

```
void lock(mutex_t *m) {
    while (test_and_set(*m)) {
        this_task.reason = m;
        this_task.state = stop;
        schedule();
    }
}
void unlock(mutex_t *m) {
    m = 0;
    foreach (i in tasks)
        if (i.state == stop && i.reason == m)
            i.state = run;
    schedule();
}
```

56

3. Mutex et Sémaphore

3.3 Problèmes associés aux sections critiques

- Voici les problèmes à prendre en considération lors de l'utilisation de sections critiques :
 - ❖ Dead Lock
 - ❖ Latence induite (ou délai de transit, ou retard)
 - ❖ Inversion de priorité

57

3. Mutex et Sémaphore

3.3.1 Dead Lock

- Aussi appelé étreinte fatale (interblocage)
- Deux tâches utilisent deux ressources imbriquées dans l'ordre inverses

tâche1 :

```
lock(m1) ;  
  
  
  
  
  
  
  
  
  
lock(m2) ;
```

tâche2 :

```
lock(m2) ;  
// dead lock ici  
  
lock(m1) ;
```

58

3. Mutex et Sémaphore

3.3.1 Dead Lock : Remarque

- Le code suivant :

```
lock(m) ;  
  
lock(m) ;
```
- entraîne un double lock, un type particulier de Dead lock

59

3. Mutex et Sémaphore

3.3.1 Dead Lock : Mutex dans une interruption

- Ne jamais utiliser de mutex dans une interruption.
- Si la ressource est occupée par la tâche qui vient d'être préemptée, le lock() s'exécutera dans le même contexte.
 - Double lock
- De plus, le blocage d'un mutex peut entraîner une très important.
- latence ce qui est en contradiction avec l'objectif de rester le minimum de temps dans une interruption.
 - Règle générale : Il ne faut pas appeler schedule dans une interruption

60

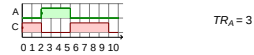
3. Mutex et Sémaphore

3.3.2 Latence

- Cas d'une tâche de faible priorité utilisant une ressource d'une tâche de haute priorité.
- Existe uniquement en environnement préemptif

Exemple avec deux tâches $P_A > P_C$:

- Cas sans partage de ressource :



- Cas avec une ressource partagée :



⁷ <http://www.sysmic.org/>

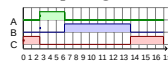
61

62

3. Mutex et Sémaphore

3.3.3 Inversion de priorité : Exemple ⁷

- On ajoute à l'exemple précédent une tâche B telle que : $P_A > P_B > P_C$
- Cas sans partage de ressource :



- Cas avec un mutex :



- La tâche B s'exécute alors qu'elle a une priorité inférieure à A et ne partage aucune ressource avec A.

⁷ <http://www.sysmic.org/>

63

64

3. Mutex et Sémaphore

3.3.3 Inversion de priorité

- Phénomène dû à la présence simultanée de priorités fixes et de ressources à accès exclusif dans un environnement préemptif.
- Une tâche de priorité intermédiaire peut être élue par l'ordonnanceur alors que des tâches de plus haute priorité attendent.
- Il est difficile de calculer le temps de blocage des tâches par une inversion de priorité (il est généralement non-borné).

3. Mutex et Sémaphore

3.3.4 Héritage de priorité

- Aussi appelé Priority inheritance
- Principe : Si une tâche bloque une ressource demandée par une tâche de plus haute priorité, elle acquiert temporairement la priorité de la tâche de haute priorité.
- L'algorithme paraît simple, mais c'est assez complexe. Il existe même des cas problématiques (dont le coût de résolution est polynomial) dans le cas d'architectures multiprocesseurs symétriques.
- Exemple précédent :



⁷ <http://www.sysmic.org/>

3. Mutex et Sémaphore

3.4 Sémaphore

- Différence entre un mutex et un sémaphore binaire : presque aucune.
- Parfois le sémaphore binaire est utilisé pour implémenter le mutex.
- Toutefois, d'un point de vue sémantique, on pourrait penser que le mutex permet d'avoir un morceau de code mutuellement exclusif tandis que le sémaphore est une section de code limité à une ressource.
- Généralement, les algorithmes d'héritage de priorité sont implémentés sur les mutex mais pas sur les sémaphore.

65

3. Mutex et Sémaphore

3.5 Mutex réentrant

- Si la même tâche tente de verrouiller le même mutex, le mutex est non-bloquant.
- Dans le cas d'un mutex non-réentrant, ceci entraîne forcément un Dead lock.
- Un sémaphore est maintenu pour connaître le nombre de passage.

66

4. Gestion de la mémoire et du temps

4.1 Gestion de la mémoire

- Le processeur va chercher les informations dont il a besoin, c'est-à-dire les instructions et les données, dans la mémoire centrale.
- La mémoire est l'assemblage de cellules repérées par leur numéro, ou adresse.
- Les cellules mémoire sont repérées par leur adresse en termes d'octets. La mémoire centrale est une ressource requise par tout processus.
- Un programme doit être chargé dans la mémoire centrale pour être exécuté.
- Mais comment gérer le partage de cette mémoire entre plusieurs processus ?

67

4. Gestion de la mémoire et du temps

4.1.1 hiérarchie de mémoire

- Du plus rapide et plus petit (niveaux bas) au plus lent et plus volumineux (niveaux hauts) :
 - ❖ Registres octets
 - ❖ Caches Kilo octets(Ko)
 - ❖ Mémoire centrale Giga octets(Go)
 - ❖ Mémoires secondaire Tera octets(To)
- Quand la capacité décroît, le coût du bit augmente.

68

4. Gestion de la mémoire et du temps

4.1.2 Techniques de partitionnement de la mémoire centrale

- Zones contiguës de taille fixées
 - ❖ Zones contiguës de taille variables
 - ❖ Zones non contiguës de taille fixe (systèmes paginés)
 - ❖ Zones non contiguës de taille variable (systèmes segmentés)

69

4. Gestion de la mémoire et du temps

4.1.3 Implémentation de la gestion de mémoire

- Pour mémoriser l'occupation de la mémoire, on utilise une liste chaînée des segments libres et occupés. Un segment est un ensemble d'unités d'allocations consécutives. Un élément de la liste est composé de quatre champs qui indiquent :
 - ❖ L'état libre ou occupé du segment.
 - ❖ L'adresse de début du segment.
 - ❖ La longueur du segment.
 - ❖ Le pointeur sur l'élément suivant de la liste.

70

4. Gestion de la mémoire et du temps

4.1.3 Implémentation de la gestion de mémoire

- Les listes peuvent être organisées en ordre :
 - ❖ de libération des zones,
 - ❖ croissant des tailles,
 - ❖ décroissant des tailles,
 - ❖ des adresses de zones.

71

4. Gestion de la mémoire et du temps

4.1.4 Allocation en zones contiguës de taille fixée

- Le système alloue aux processus des zones mémoires de tailles supérieure ou égale à leur demande
- Problèmes liés à la gestion de la mémoire en partitions :
 - ❖ **Fragmentation interne** : Un processus n'utilise pas la totalité de la partition.
 - ❖ **Fragmentation externe** : Certaines partitions ne sont pas utilisées, car trop petites pour tous les travaux éligibles.

72

4. Gestion de la mémoire et du temps

4.1.5 Allocation en zones contiguës de taille variables

- Le système alloue aux processus des zones mémoires de tailles égale à leur demande.
- Stratégies d'allocation :
 - ❖ First-Fit: on place le processus dans la première partition dont la taille est suffisante. liste non ordonnée, ou ordonnée par adresse
 - ❖ Best-Fit: on place le processus dans la partition dont la taille est la plus proche de la sienne. liste ordonnée par tailles croissante
 - ❖ Worst-Fit: on place le processus dans la partition dont la taille est la plus grande. liste ordonnée par tailles décroissantes

73

4. Gestion de la mémoire et du temps

4.2. La pagination : Mémoire virtuelle

- Soit M la taille de la mémoire disponible sur la machine. Soit T la taille de l'application :
 - ❖ Si $T < M$, on parle simplement de pagination,
 - ❖ Si $T > M$, on parle de mémoire virtuelle,
- La mémoire virtuelle donne l'illusion à l'utilisateur (au processus) qu'il dispose d'un espace d'adressage illimité (en fait, limité par la taille du disque).
- Lorsqu'on ne trouve pas la page que l'on cherche en mémoire, on parle de défaut de page (page fault, en anglais).

75

4. Gestion de la mémoire et du temps

4.2 La pagination

- Le problème des partitions est la fragmentation car les programmes ont besoin d'espace contigu. La pagination permet d'utiliser des espaces non contigus.
- L'espace d'adressage est divisé en pages, la mémoire « centrale » est divisée en blocs. (taille de la page = taille du bloc).
- Table des pages : elle fait correspondre une page logique à un bloc physique.
- Quand l'espace d'adressage de l'application est supérieur à la taille de la mémoire physique, on parle d'adressage virtuel.

74

4. Gestion de la mémoire et du temps

4.3 Algorithmes de remplacement

- OPTIMAL
- LRU (Last-Recently Used) : la page la moins récemment utilisée: l'algorithme le plus utilisé.
- Least Frequently Used (LFU), la page la moins fréquemment utilisée.
- FIFO First In First Out (FIFO), la plus ancienne.
- Seconde chance.

76

4. Gestion de la mémoire et du temps

4.4 Gestion du temps

- Horloges et mesure du temps
- Deux types d'horloges :
 - ❖ Horloge du CPU (ticks) (TSC, HPET,...)
 - ❖ Horloge murale (temps universel)
- Variations de la fréquence de l'électronique : → écarts entre les deux.
- NTP (Network Time Protocol) :
- protocole de synchronisation des horloges via internet sur temps universel.

77

4. Gestion de la mémoire et du temps

4.4.1 Temps universel POSIX

```
#include <time.h>
```

```
time_t time(time_t *tloc);
```

`time(NULL)` retourne le nombre de secondes écoulées depuis le 1er janvier 1970 à 00 :00 UTC.

`time_t` est un entier signé 32 bits. (débordement le 19 janvier 2038)

le noyau utilise l'heure UTC. Gestion du fuseau horaire (timezone) dans les bibliothèques.

78

4. Gestion de la mémoire et du temps

```
#include <sys/types.h>
#include <time.h>

struct tm *localtime_r(const time_t *clock, struct tm* tp);
time_t mktime(struct tm* t);

struct tm {
    int tm_sec; /* seconds (0 - 60) */
    int tm_min; /* minutes (0 - 59) */
    int tm_hour; /* hours (0 - 23) */
    int tm_mday; /* day of month (1 - 31) */
    int tm_mon; /* month of year (0 - 11) */
    int tm_year; /* year - 1900 */
    int tm_wday; /* day of week (Sunday = 0) */
    int tm_yday; /* day of year (0 - 365) */
    int tm_isdst; /* is summer time in effect? */
    long tm_gmtoff; /* offset from UTC in seconds */
    char *tm_zone; /* abbreviation of timezone name */
}
```

79

4. Gestion de la mémoire et du temps

4.4.2 Horloge POSIX

```
#include <time.h>
```

```
int clock_gettime(clockid_t, struct timespec *tp);
```

```
struct timespec {
    time_t tv_sec; /* seconds */
    long tv_nsec; /* nanoseconds */
};
```

```
clockid_t :
```

```
CLOCK_REALTIME temps universel ;
```

```
CLOCK_MONOTONIC horloge CPU ;
```

```
CLOCK_PROCESS_CPUTIME_ID horloge du processus ;
```

80

4. Gestion de la mémoire et du temps

4.4.3 Timers

- Mécanisme de déclenchement d'une interruption (signal) dans le futur.
 - ❖ périodique ou non.
 - ❖ exécution d'une fonction associée au moment du déclenchement.
 - ❖ contexte interruption : sections critiques, fonctions interdites.
 - ❖ peuvent être associés à différentes horloges.
- Utilisations :
 - ❖ Permettent le cadencement précis d'un traitement périodique.
 - ❖ Mécanisme de garde (watchdog) pour traitements bloquants.

81

4. Gestion de la mémoire et du temps

4.4.3 Timers POSIX

```
#include <signal.h>
#include <time.h>
int timer_create(clockid_t clockid, struct sigevent *sevp, timer_t *timerid);
int timer_settime(timer_t timerid, int flags,
                  const struct itimerspec *new_value,
                  struct itimerspec *old_value);

struct itimerspec {
    struct timespec it_interval; /* Timer interval */
    struct timespec it_value; /* Initial expiration */
};
```

➤ sevp définit le signal qui est déclenché (SIGALRM) signal(SIGALRM, fonction) permet de définir la fonction qui sera appelée

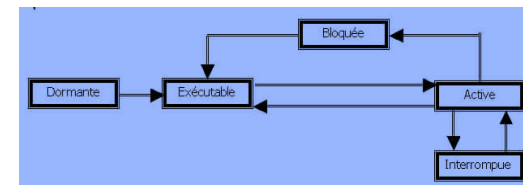
82

5. Les interruptions

- Une tâche est un programme qui s'exécute comme s'il était le seul à utiliser le CPU. Il possède une priorité, du code à effectuer, une partie de la mémoire et une zone de pile.
- Une tâche peut être dans différents états:
 - ❖ Dormante : La tâche est en mémoire mais n'est pas considérée par l'ordonnanceur,
 - ❖ Exécutable : La tâche est prête à être exécutée mais n'a pas le CPU,
 - ❖ Active : La tâche est exécutée par le CPU,
 - ❖ Bloquée : La tâche est en attente d'un signal ou d'une ressource pour poursuivre,
 - ❖ Interrompue : La tâche est suspendue par une interruption.

83

5. Les interruptions



84

5. Les interruptions

Etat d'origine	Nouvel Etat	Evènement provoquant la transition
Dormante	Exécutable	La tâche est activée
Exécutable	Active	L'Ordonnanceur donne le CPU à la tâche
Active	Bloquée	La tâche est en attente d'une ressource (bloquée par un sémaphore).
Bloquée	Exécutable	La ressource dont la tâche était en attente est libérée
Active	Exécutable	L'Ordonnanceur donne le CPU à une autre tâche (préemption)
Active	Interrompue	Une interruption stoppe la tâche
Interrompue	Active	Le traitement de l'interruption est terminé

85

5. Les interruptions

- Quand l'ordonnanceur doit transférer l'usage du processeur d'une tâche à l'autre, il opère un changement de contexte. Le contexte représente l'état du processeur à un moment donné.
- La tâche suspendue doit pouvoir continuer son exécution sans être affectée, la première opération à effectuer est donc la sauvegarde de l'état du processeur au moment de la suspension. Le noyau possède pour chaque tâche non dormante un espace mémoire réservé à cet effet.

87

5. Les interruptions

- L'Ordonnanceur est le composant du noyau responsable de la distribution du temps CPU entre les différentes tâches.
- Il est basé sur le principe de la priorité : chaque tâche possède une priorité dépendant de son importance (plus une tâche est importante, plus sa priorité est élevée).
- Il attribue le processeur à la tâche exécutable (non dormante et non bloquée) de plus haute priorité.
- Lorsque qu'une tâche T2 plus prioritaire que la tâche active T1 passe de l'état bloqué à l'état exécutable, T1 est suspendue et l'Ordonnanceur attribue le processeur à T2.
- C'est le principe de la préemption. La préemption permet un temps de réponse des tâches optimum.

86

5. Les interruptions

- Changement de contexte.
- Chaque tâche possède une pile en mémoire. Une liste globale contient :
 - ❖ les états de toutes les tâches,
 - ❖ l'emplacement de la pile en mémoire,
 - ❖ le contexte d'exécution, c'est-à-dire une sauvegarde des registres.
- Lors du changement de contexte
 - ❖ on sauvegarde le contexte de la tâche précédente, en particulier son pointeur de pile et son pointeur d'instruction,
 - ❖ on restaure le contexte de la nouvelle tâche,
 - ❖ on restaure le pointeur d'instruction.
- Dans la pratique, il y a des petites subtilités selon la manière dont le changement de contexte à été amené.

88

5. Les interruptions

- Multitâches non-préemptif
- Le changement de contexte peut-être volontaire par les tâches. Dans ce cas, la tâche ayant terminé son traitement appellera explicitement la fonction schedule qui effectuera la changement de contexte.
- Le système est dit non-préemptif ou multitâches collaboratif.

89

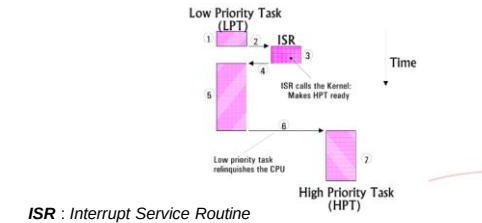
5. Les interruptions

- Multitâches non-préemptif
- 1) Une tâche non prioritaire est en cours d'exécution et est interrompue par un évènement IRQ (Interrupt ReQuest ou interruption matérielle)
 - 2) L'ISR est appelée
 - 3) Le traitement l'IRQ rend une tâche de haute priorité prête à être exécutée
 - 4) A la fin de l'ISR, le système rend le CPU à la tâche non-prioritaire
 - 5) Quand la tâche non-prioritaire termine son traitement, elle appelle schedule
 - 6) L'ordonnanceur donne la main à la tâche de forte priorité
 - 7) La tâche de haute priorité peut (enfin) s'exécuter

91

5. Les interruptions

- Multitâches non-préemptif
- Ce type de système implique une latence difficilement quantifiable entre un évènement et son traitement :



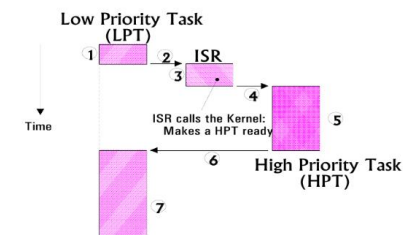
¹ <https://moodle.insa-toulouse.fr/>

90

5. Les interruptions

- Multitâches préemptif

Un système multitâches préemptif va être capable de changer de contexte lors des interruptions :



¹ <https://moodle.insa-toulouse.fr/>

92

5. Les interruptions

- Multitâches préemptif

- 1) Une tâche non prioritaire est en cours d'exécution et est interrompue par un événement (une IRQ).
- 2) L'ISR est rappel.
- 3) Le traitement l'IRQ rend une tâche de haute priorité prête à être exécutée.
- 4) A la fin de l'ISR, le système appelle le scheduler.
- 5) Le scheduler donne la main à la tâche de haute priorité.
- 6) Quand la tâche prioritaire termine son traitement, elle appelle schedule.
- 7) Vu qu'il n'y a plus tâche prioritaire à exécuter, l'ordonnanceur redonne la main à la tâche de faible priorité.

93

webographie

¹ <https://moodle.insa-toulouse.fr/>

² wikipedia.org

³ techniques-ingénieur.fr

⁴ <https://moodle.insa-toulouse.fr/>

⁵ <http://www.mnis.fr/>

⁶ <https://sites.uclouvain.be/>

⁷ <http://www.sysmic.org/>

94